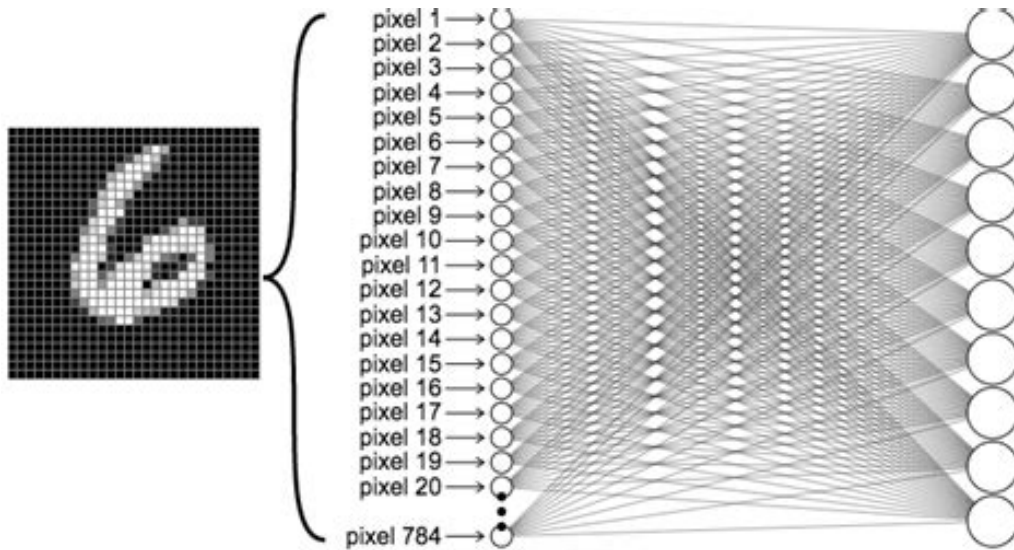


SpikeLens

脉冲神经网络推理过程的交互式可视化

Big Data Visualization and Visual Analysis Final Project



项目总览与可视化界面截图。

项目网页: https://c1truus.github.io/spikelens_bdva/

代码仓库: https://github.com/c1truus/spikelens_bdva

作者: 蒙和特尔学号: 1820221121

报告类型: README 风格技术项目报告

目录

1 项目概述	4
1.1 摘要	4
1.2 核心可视化问题	4
1.3 最终成果	4
2 项目动机与想法形成	4
2.1 初始方向	5
2.2 为什么需要可视化 SNN 推理	5
2.3 为什么采用静态 Web 应用	5
2.4 设计约束	6
3 背景知识	6
3.1 MNIST 数据集	6
3.2 脉冲神经网络	6
3.3 泄漏积分发放神经元	7
3.4 延迟编码	7
3.5 脉冲序列	8
3.6 项目中使用的超参数	9
3.6.1 Beta	9
3.6.2 Latency Threshold	9
3.6.3 Inference Timesteps	9
4 数据可视化问题	9
4.1 主问题	9
4.2 子问题	9
4.3 为什么普通静态图不够	10
4.4 视觉抽象策略	10
5 模型与数据生成	10
5.1 网络拓扑	10
5.2 超参数扫描	11
5.3 训练流程	11
5.4 评估流程	11
5.5 轨迹导出流程	11
5.6 JSON 轨迹结构	12
5.7 Manifest 文件	12
5.8 部署数据裁剪	13

6 可视化设计	13
6.1 主网络视图	13
6.2 输入层	13
6.3 隐藏层	14
6.4 输出层	14
6.5 颜色模式	14
6.6 层脉冲时间线	14
6.7 延迟编码解释面板	14
6.8 超参数解释面板	15
7 Web 应用架构	15
7.1 技术栈	16
7.2 目录结构	16
7.3 静态数据加载	16
7.4 组件结构	17
8 功能总结	17
8.1 模型参数选择	17
8.2 样本选择	17
8.3 播放与拖动	17
8.4 多视图联动	17
9 本地运行与部署	18
9.1 本地运行	18
9.2 生产构建	18
9.3 GitHub Pages 部署	18
10 可复现性说明	18
10.1 训练 notebook	18
10.2 导出 notebook	18
10.3 Manifest 生成脚本	19
10.4 仓库中包含与不包含的内容	19
11 局限性	19
11.1 只展示预计算轨迹	19
11.2 发布样本数量有限	19
11.3 JSON 文件较大	19
11.4 隐藏层网格是抽象布局	20
12 未来工作	20
12.1 压缩轨迹格式	20
12.2 更多模型与样本	20

12.3 在线推理后端	20
12.4 模型统计面板	20
12.5 FPGA 轨迹对比	20
13 总结	20
14 参考资料	21

1 项目概述

1.1 摘要

SpikeLens 是一个用于展示脉冲神经网络推理过程的交互式静态网页应用。项目以 MNIST 手写数字识别为任务，将静态的灰度数字图像通过延迟编码转换为输入脉冲序列，然后记录脉冲神经网络在多个时间步内的输入层脉冲、隐藏层脉冲、膜电位以及输出类别脉冲计数，并将这些记录可视化可播放、可拖动、可比较的时间序列界面。

本项目的目标不是单纯展示一个分类准确率，而是打开神经网络推理过程本身：让观察者看到一个静态数字如何被编码成脉冲序列，脉冲如何在隐藏层中传播，不同类别输出神经元如何逐步积累证据，以及不同超参数如何影响网络内部的可视化动态。

最终系统采用静态网站形式部署。浏览器端不运行 PyTorch 或 snnTorch 模型，而是加载离线预先导出的 JSON 推理轨迹数据。这种设计降低了部署复杂度，使项目可以直接通过 GitHub Pages 访问，并保持较好的可复现性与可展示性。

1.2 核心可视化问题

本项目围绕以下核心问题展开：

脉冲神经网络如何从延迟编码的手写数字中逐步积累时间证据，并且膜衰减系数、输入延迟阈值和推理时间步数等超参数如何改变脉冲传播与膜电位变化的可视化形态？

这个问题区别于普通分类任务中的“预测是否正确”。本项目更关注导致分类结果的内部时序过程。

1.3 最终成果

项目最终包含以下成果：

- 一组经过训练的脉冲神经网络模型变体；
- 由模型推理过程导出的脉冲与膜电位轨迹数据；
- 基于 `manifest.json` 的静态数据索引结构；
- 使用 Svelte 与 Vite 构建的交互式网页应用；
- 包含输入脉冲、隐藏层状态、膜电位强度、输出脉冲累积、时间步播放、延迟编码解释和超参数解释的可视化界面。

2 项目动机与想法形成

2.1 初始方向

本项目的想法来源于脉冲神经网络硬件加速相关实践。在该方向中，模型推理不再只是一次矩阵运算，而是涉及时间步、脉冲事件、膜电位累积、延迟编码以及层间事件传播。相比普通神经网络，脉冲神经网络天然具有时间结构，因此更适合作为一个可视分析对象。

普通人工神经网络通常可以被理解为：

```
input image -> output class
```

而脉冲神经网络更接近：

```
input image -> spike sequence -> layer activity over time -> output spike counts
```

这种从静态输入到时序事件的转换，构成了本项目最核心的可视化机会。

2.2 为什么需要可视化 SNN 推理

脉冲神经网络的内部状态不仅由最终输出决定，还由离散脉冲事件和膜电位变化共同构成。一个神经元可能在多个时间步内保持静默，同时不断积累膜电位；当膜电位超过阈值时，它才会发放一个二值脉冲。

因此，推理过程中存在多层信息：

- 哪些输入像素较早发放脉冲？
- 隐藏层中哪些神经元被激活？
- 网络活动在时间上是否稀疏？
- 哪个输出类别神经元积累了最多脉冲？
- 超参数变化如何影响可见的传播模式？

这些问题很难仅通过准确率或混淆矩阵回答。SpikeLens 的作用是把一次 SNN 推理过程转化为可观察、可交互、可解释的视觉对象。

2.3 为什么采用静态 Web 应用

实时后端可以支持在线推理，但会显著增加部署成本。对于课程期末项目，更重要的是让教师可以直接打开网页、稳定访问并体验交互。因此，本项目采用如下结构：

```
offline training and trace export
  -> JSON trace dataset
  -> static Svelte/Vite web app
  -> GitHub Pages deployment
```

浏览器端只加载预先计算好的数据，不依赖本地 Python、PyTorch、CUDA 或 snnTorch 环境。这样可以保证展示过程更稳定，也更适合提交与评分。

2.4 设计约束

项目实施过程中主要考虑以下约束：

- 网站应能在无后端环境下运行；
- 数据大小必须适合 GitHub Pages 部署；
- 可视化必须在数百到数千个神经元规模下仍然可读；
- 交互方式必须对第一次接触 SNN 的读者足够清晰；
- 项目不应只展示图像，还应解释 SNN 的基本概念。

3 背景知识

3.1 MNIST 数据集

MNIST 是手写数字分类任务中最经典的数据集之一。每个样本是一张 28×28 的灰度图像，对应 0 到 9 中的一个数字类别。该数据集的标准版本包含训练集和测试集，常用于机器学习与神经网络模型的教学、验证和可视化实验。

本项目选择 MNIST 的原因有两点。第一，手写数字具有直观含义，读者可以直接比较原始图像、真实标签与预测标签。第二， 28×28 的输入尺寸可以自然映射成二维网格，适合展示输入层脉冲活动。

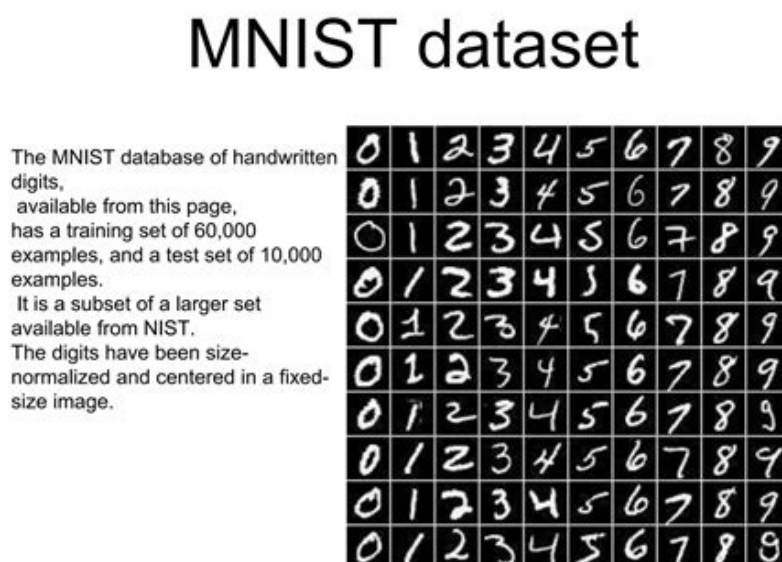


图 1: MNIST 输入图像示例。每个像素可以被视为一个输入神经元。

3.2 脉冲神经网络

脉冲神经网络使用离散脉冲事件表示信息。与普通人工神经网络中连续的激活值不同，SNN 中的神经元在特定时间步发放二值脉冲。也就是说，模型的计算结果不仅与“值”有关，也与

“时间”有关。

可以简化理解为：

```
ordinary neural network: continuous activation values
spiking neural network: binary events over discrete time
```

这种时间维度使 SNN 的推理过程更适合被做成动画或交互式时间序列可视化。

3.3 泄漏积分发放神经元

本项目使用 `snnTorch` 中的 Leaky Integrate-and-Fire 行为。LIF 神经元维护一个膜电位状态。输入电流会提高膜电位，膜电位会随着时间按比例衰减。当膜电位超过发放阈值时，神经元输出一个脉冲。

其直观过程为：

```
input current arrives
  -> membrane potential accumulates
  -> old potential decays by beta
  -> threshold crossing emits a spike
```

在可视化中，膜电位被显示为亮度强度。膜电位越高，颜色越亮；膜电位越低，颜色越暗。

3.4 延迟编码

MNIST 图像本身是静态的，而 SNN 的输入是时间序列。为了解决这个问题，本项目使用延迟编码将像素亮度映射为脉冲时间。

延迟编码的直观含义是：

- 较亮的像素更早发放脉冲；
- 较暗的像素更晚发放脉冲；
- 低于阈值的弱像素可以不发放脉冲；
- 一张静态图像被转换为多个时间步上的输入脉冲帧。

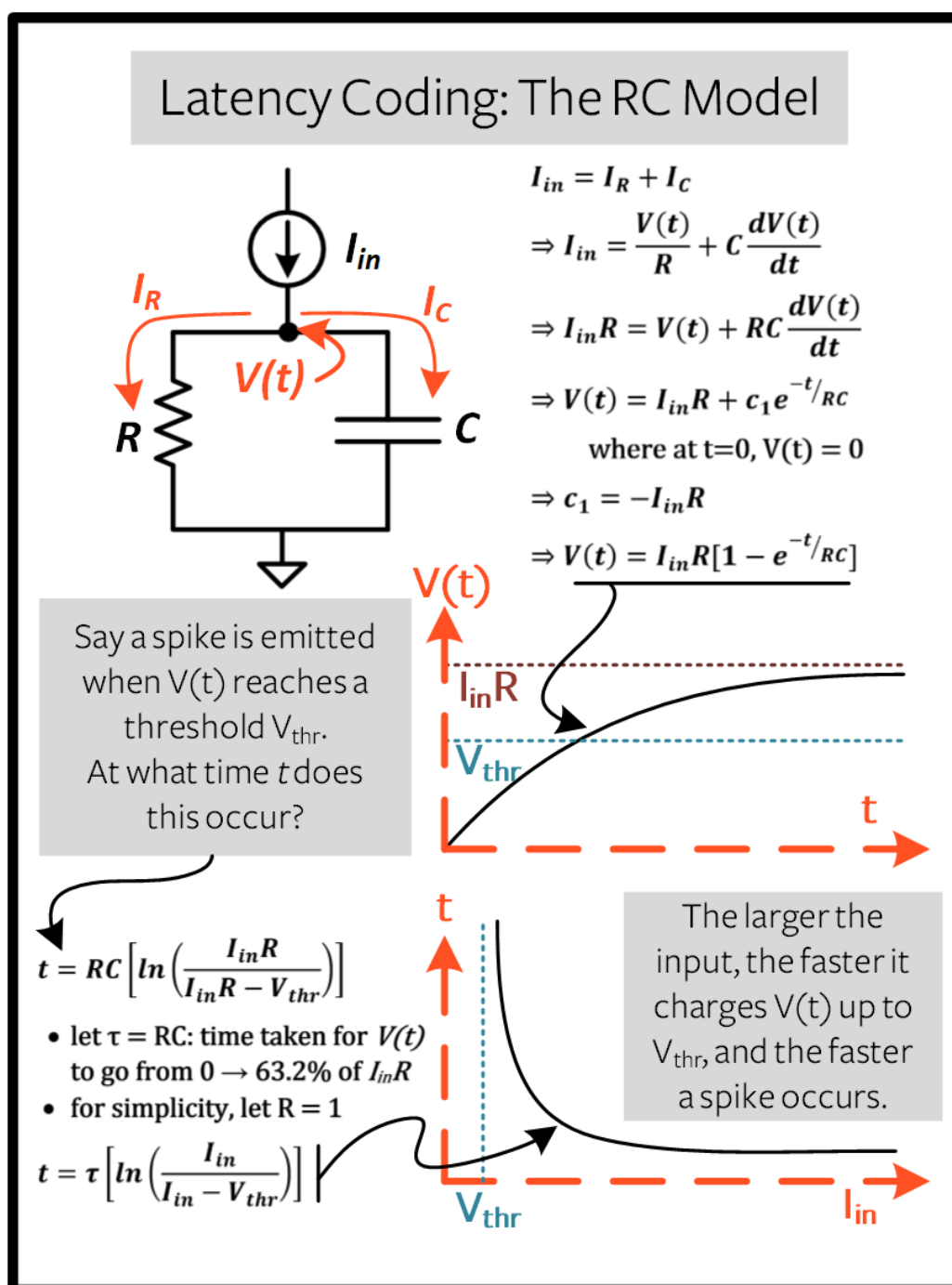


图 2: 延迟编码示意图。静态图像被转换为一段输入脉冲序列。

3.5 脉冲序列

单个神经元的脉冲序列可以看作一个由 0 和 1 构成的时间数组：

```

timestep: 0 1 2 3 4 5 ...
spike:    1 0 0 1 0 0 ...
  
```

对于 MNIST 输入层，784 个像素对应 784 个输入神经元。因此，一张图像经过编码后会形成 784 条输入脉冲序列。

3.6 项目中使用的超参数

3.6.1 Beta

Beta 表示膜电位记忆能力。较大的 beta 代表神经元保留更多过去的膜电位，较小的 beta 代表膜电位衰减更快。

```
low beta -> short memory  
high beta -> longer memory
```

3.6.2 Latency Threshold

Latency threshold 控制哪些输入像素强度足以被编码为脉冲。阈值越高，越多弱像素被过滤掉，输入序列越稀疏。

```
low threshold -> more pixels can spike  
high threshold -> fewer and stronger pixels spike
```

3.6.3 Inference Timesteps

推理时间步数决定网络在多少帧内积累证据。较少时间步意味着推理更短；较多时间步意味着模型有更长的证据积累窗口。

```
fewer steps -> shorter evidence window  
more steps -> longer temporal evidence
```

4 数据可视化问题

4.1 主问题

本项目的主问题是：

延迟编码后的视觉证据如何在脉冲神经网络中随时间传播？

该问题关注推理过程本身，而不仅仅是最终的分类结果。

4.2 子问题

SpikeLens 被设计用来帮助回答以下问题：

1. 输入数字的哪些像素最早发放脉冲？
2. 输入脉冲序列的稀疏程度如何？
3. 隐藏层活动如何随时间变化？
4. 哪个输出类别神经元积累最多脉冲？

5. 正确类别是在早期还是后期开始占优?
6. beta、latency threshold 和 timestep 数量如何改变可见动态?
7. 如何在不绘制所有全连接边的情况下表示大规模神经网络状态?

4.3 为什么普通静态图不够

普通静态图可以展示某一帧或某个统计量，但无法完整表达 SNN 推理的时序结构。读者需要能够拖动时间轴、播放推理过程、切换模型参数、选择样本，并观察多个层的协调变化。因此，交互式网页比单张图像更适合本项目。

4.4 视觉抽象策略

全连接网络包含大量边。如果直接绘制所有突触连接，画面会被边线淹没，难以理解。因此，SpikeLens 不绘制每条连接，而是将每一层抽象为二维网格：

```
Input layer: 28 x 28
Hidden layer 1: 32 x 32
Hidden layer 2: 16 x 16
Output layer: 1 x 10
```

这种设计保留了神经元数量和层级结构，同时避免了边连接造成的视觉噪声。

5 模型与数据生成

5.1 网络拓扑

本项目发布版使用的模型拓扑为：

```
784 -> 1024 -> 256 -> 10
```

该结构与可视化布局自然对应：

表 1: 网络层与可视化映射。

层	神经元数量	可视化形式
输入层	784	28 × 28 网格
FC1 隐藏层	1024	32 × 32 网格
FC2 隐藏层	256	16 × 16 网格
输出层	10	10 个类别单元

选择 1024 与 256 的重要原因是它们可以形成规则方形网格。相比 512 这种不方便构成方阵的大小，方形网格更适合视觉展示。

5.2 超参数扫描

项目训练了 36 个模型变体，参数组合为：

```
beta: 0.65, 0.75, 0.85, 0.95
latency threshold: 0.15, 0.25, 0.35
timesteps: 25, 40, 60
```

组合数量为：

$$4 \times 3 \times 3 = 36 \quad (1)$$

这个规模既能支持参数比较，又不会让网页交互变得过于复杂。

5.3 训练流程

训练流程在浏览器外部完成。训练 notebook 的主要步骤包括：

1. 加载 MNIST 数据集；
2. 定义 SNN 模型结构；
3. 将输入图像编码为脉冲序列；
4. 对每个超参数组合训练模型；
5. 在测试集上进行评估；
6. 保存模型权重、checkpoint、训练历史、metadata 和完成标记。

这些训练产物用于生成推理轨迹，但不会全部上传到网页仓库。网页只需要导出的 JSON 轨迹数据。

5.4 评估流程

每个模型在训练结束后都会进行测试集评估。评估结果包括预测类别、真实类别以及是否预测正确。这些信息被写入导出的推理轨迹和 manifest 中，用于前端显示当前样本是否分类正确。

5.5 轨迹导出流程

第二个 notebook 负责加载训练好的模型，并对选定的 MNIST 测试样本进行推理记录。每条轨迹包含：

- 原始输入图像；
- 真实标签与预测标签；
- 每个时间步的输入层脉冲；
- FC1 和 FC2 隐藏层脉冲；

- 输出层脉冲;
- 各层膜电位;
- 输出类别累计脉冲计数;
- 与模型相关的超参数 metadata。

5.6 JSON 轨迹结构

每个样本被保存为一个 JSON 文件:

```
spikelens_webapp/public/data/traces/<model_id>/<sample>.json
```

简化结构如下:

```
{
  "sample_idx": 0,
  "true_label": 7,
  "pred_label": 7,
  "correct": true,
  "num_steps": 25,
  "beta": 0.65,
  "latency_threshold": 0.15,
  "original_image": [...],
  "spikes_dense": {
    "input": [...],
    "fc1": [...],
    "fc2": [...],
    "out": [...]
  },
  "membrane": {
    "fc1": [...],
    "fc2": [...],
    "out": [...]
  },
  "output_spike_counts": [...]
}
```

实际文件更大, 因为其中保存了完整的时间步和层状态数组。

5.7 Manifest 文件

由于 GitHub Pages 是静态托管, 浏览器不能动态扫描服务器目录。因此项目使用:

```
spikelens_webapp/public/data/manifest.json
```

该文件记录所有可用模型、超参数、样本路径、预测标签与统计信息。前端首先加载 manifest, 再根据用户选择加载具体 JSON 轨迹。

5.8 部署数据裁剪

完整轨迹数据体积较大，尤其是膜电位数组会显著增加 JSON 文件大小。因此发布版本只保留精简子集：

```
36 model variants x 5 samples per model = 180 traces
```

这种规模能够保证网页可用，同时仍然保留足够的模型和样本多样性。

6 可视化设计

6.1 主网络视图

主可视化视图将 SNN 表示为层级空间布局：

```
Input 28x28 -> FC1 32x32 -> FC2 16x16 -> Output classes 0-9
```

该布局不是严格的数学图，而是带有轻微空间透视感的层状态展示。其目的不是展示每条权重连接，而是让读者观察不同层的时间状态变化。



图 3: 主网络轨迹视图。输入层、隐藏层与输出层在同一界面中同步显示。

6.2 输入层

输入层显示当前时间步下的延迟编码脉冲帧。高亮单元表示该像素神经元在当前时间步发放脉冲。通过该视图可以观察数字笔画中哪些位置最早参与推理。

6.3 隐藏层

隐藏层分别显示为 32×32 和 16×16 网格。这些网格是神经元编号的视觉映射，不代表物理布局或卷积特征图。其主要作用是使大规模全连接层状态能够被读者快速观察。

6.4 输出层

输出层包含 10 个类别单元，对应数字 0 到 9。输出图表显示截至当前时间步的累计输出脉冲数。预测类别由累计脉冲数最大的输出神经元决定。

6.5 颜色模式

界面提供三种颜色模式：

Spike mode 显示当前时间步的二值脉冲事件，用于观察“此刻哪些神经元发放了脉冲”。

Membrane mode 显示膜电位强度，用于观察“哪些神经元内部状态较强或接近发放”。

Cumulative mode 显示累计活动，用于观察“整个推理过程中哪些神经元较常活跃”。

6.6 层脉冲时间线

时间线图展示不同层在每个时间步的脉冲数量。读者可以通过该图观察输入脉冲爆发、隐藏层活动波动以及输出层脉冲积累过程。

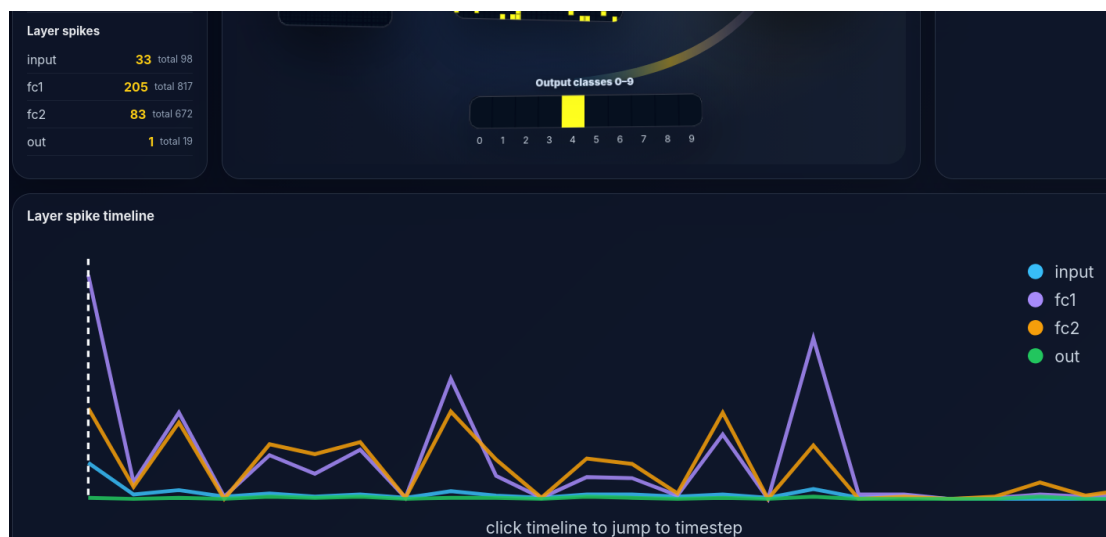


图 4: 层脉冲时间线。该图帮助读者理解不同层活动随时间的变化。

6.7 延迟编码解释面板

为了让读者理解静态图像如何变成脉冲序列，项目加入了专门的延迟编码解释面板。该面板显示：

- 原始数字图像；

- 首次脉冲时间图；
- 当前时间步输入脉冲帧；
- 从第 0 步到当前时间步的累计输入显示；
- 每个时间步的输入脉冲数量。

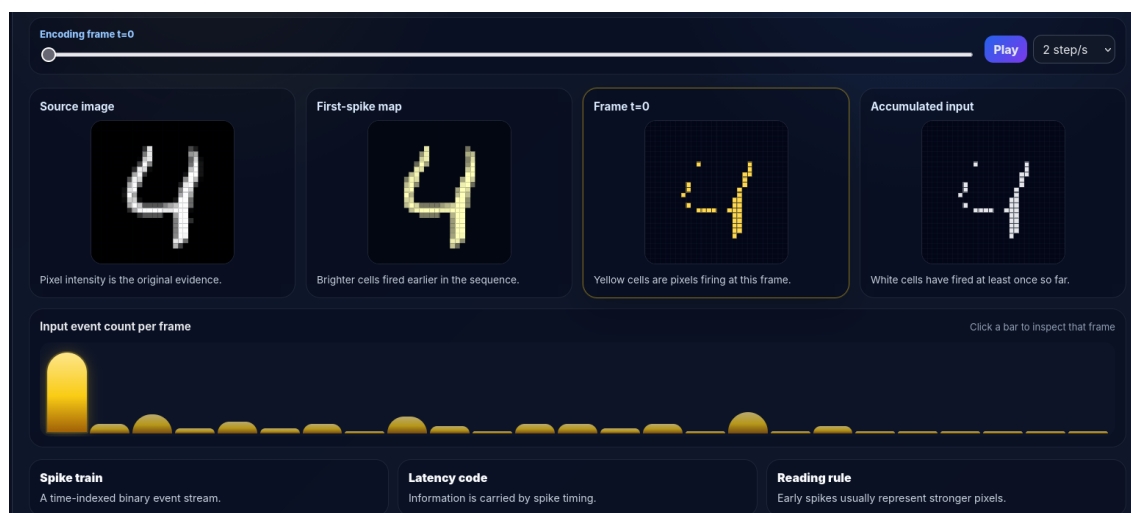


图 5: 延迟编码解释面板。该部分将输入图像、首次脉冲时间、当前脉冲帧和累计输入统一展示。

6.8 超参数解释面板

超参数解释面板用较清晰的语言解释 β 、latency threshold 和 inference timesteps 对模型行为的影响。该面板的目的不是展示公式，而是帮助非专业读者理解参数的实际作用。



图 6: 超参数解释面板。beta 表示膜记忆，latency threshold 表示输入筛选强度，timesteps 表示证据积累窗口。

7 Web 应用架构

7.1 技术栈

表 2: 项目技术栈。

层级	技术	用途
模型训练	Python, PyTorch, snnTorch	训练 SNN 并记录推理轨迹
数据格式	JSON	静态轨迹存储
前端框架	Svelte	构建交互式组件
构建工具	Vite	生成静态网站文件
可视化	Canvas, SVG, D3	绘制层网格、图表和时间线
部署	GitHub Pages	静态网页托管

7.2 目录结构

项目目录结构如下：

```
spikelens_bdva/  
  README.md  
  spikelens_webapp/  
    index.html  
    package.json  
    vite.config.js  
    public/  
      data/  
        manifest.json  
      traces/  
    src/  
    App.svelte  
    main.js  
    styles.css  
    components/  
    lib/  
  tools/  
    build_spikelens_manifest.py
```

7.3 静态数据加载

前端加载流程为：

```
fetch /data/manifest.json  
  -> populate model and sample selectors  
  -> fetch selected trace JSON  
  -> render visualization
```

该流程不需要后端 API。所有数据都位于 `public/data` 中，并在 Vite 构建时复制到最终静态网站目录。

7.4 组件结构

主要 Svelte 组件包括：

表 3: 前端组件与功能。

组件	功能
<code>App.svelte</code>	主应用状态与整体布局
<code>NetworkScene.svelte</code>	主网络轨迹可视化
<code>DigitPreview.svelte</code>	输入数字与脉冲预览
<code>OutputChart.svelte</code>	输出类别脉冲图表
<code>TimelineChart.svelte</code>	层脉冲时间线
<code>LayerSummary.svelte</code>	当前时间步的层统计
<code>LatencyEncodingExplainer.svelte</code>	延迟编码教学面板
<code>HyperparameterExplainer.svelte</code>	超参数教学面板
<code>ResourceNotes.svelte</code>	资源与引用说明

8 功能总结

8.1 模型参数选择

用户可以分别选择 `beta`、`latency threshold` 和 `inference steps`。相比把所有模型变体放在一个很长的下拉菜单中，这种方式更符合参数比较的逻辑。

8.2 样本选择

用户可以选择当前模型下的输入样本。界面会显示真实标签、预测标签以及是否分类正确。

8.3 播放与拖动

时间步可以通过滑块拖动，也可以通过播放按钮自动前进。这样一次 SNN 推理可以被观看为短动画。

8.4 多视图联动

当前时间步会同步影响：

- 输入脉冲帧；
- 隐藏层状态；
- 输出类别图表；

- 时间线标记;
- 延迟编码解释面板。

9 本地运行与部署

9.1 本地运行

运行网页应用:

```
cd spikelens_webapp
npm install
npm run dev
```

9.2 生产构建

```
cd spikelens_webapp
npm run build
npm run preview
```

9.3 GitHub Pages 部署

对于项目页部署, Vite 的 base path 应设置为仓库名:

```
export default defineConfig({
  plugins: [svelte()],
  base: '/spikelens_bdva/'
});
```

构建后发布 dist/ 到 gh-pages 分支即可。

10 可复现性说明

10.1 训练 notebook

训练 notebook 负责定义模型、加载 MNIST、执行超参数扫描、训练模型、评估模型并保存训练历史。训练产物包括模型权重、checkpoint、metadata 和日志。

10.2 导出 notebook

导出 notebook 负责加载训练好的模型, 对测试样本进行推理, 记录各层脉冲和膜电位, 并写入 JSON 文件。

10.3 Manifest 生成脚本

`tools/build_spikelens_manifest.py` 会扫描导出的轨迹目录, 生成 `spikelens_webapp/public/data/manifest.json`。前端依赖该文件发现可用模型和样本。

10.4 仓库中包含与不包含的内容

发布仓库包含:

- 前端源代码;
- 精简后的 JSON 推理轨迹;
- manifest 文件;
- 报告与说明文档;
- GitHub Pages 部署所需文件。

发布仓库不包含:

- 原始 MNIST 数据;
- PyTorch 模型 checkpoint;
- Python 虚拟环境;
- `node_modules`;
- 完整本地轨迹数据。

这些内容对于在线展示不是必需的, 并且会显著增加仓库体积。

11 局限性

11.1 只展示预计算轨迹

网页端不会实时运行模型。它只能展示已经导出的样本轨迹。如果需要用户上传数字并实时推理, 需要增加后端服务。

11.2 发布样本数量有限

为了适配静态托管与文件体积限制, 发布版本只保留部分样本。完整本地数据可以更大, 但不适合全部部署到 GitHub Pages。

11.3 JSON 文件较大

为了支持膜电位可视化, 轨迹中保存了较密集的数组。未来可以使用稀疏事件格式、二进制格式或压缩方式降低体积。

11.4 隐藏层网格是抽象布局

隐藏层的 32×32 与 16×16 网格只是神经元编号的视觉布局，不表示卷积特征图或真实物理拓扑。

12 未来工作

12.1 压缩轨迹格式

未来可以将 dense JSON 改为稀疏事件记录或二进制数组，以减少下载体积并提升加载速度。

12.2 更多模型与样本

当前发布版本是精简数据集。未来可以使用后端或对象存储支持更多模型变体和更多样本。

12.3 在线推理后端

如果加入后端，可以支持用户上传自己的数字图像，并实时生成对应 SNN 推理轨迹。

12.4 模型统计面板

未来可以增加模型级统计视图，例如不同 beta 下的平均准确率、平均脉冲数、稀疏度和输出置信度。

12.5 FPGA 轨迹对比

由于项目灵感来自 SNN 硬件加速方向，未来可以进一步比较软件 snnTorch 轨迹与 FPGA 硬件轨迹，从而将可视分析扩展到硬件验证。

13 总结

SpikeLens 将一次脉冲神经网络推理过程转化为可交互的时序可视化。项目的主要贡献不是提出新的模型结构，而是构建了一个完整的可视分析流程：

```
train SNN models
record inference dynamics
structure traces as static data
visualize temporal propagation interactively
explain encoding and hyperparameters to readers
```

通过该流程，SNN 的内部事件、膜电位变化和输出证据积累过程变得更加可见、可检查和可解释。

14 参考资料

1. snnTorch documentation: spike generation and conversion methods, including latency coding and rate coding.
<https://snntorch.readthedocs.io/en/latest/snntorch.spikegen.html>
2. snnTorch documentation: `snn.Leaky`, first-order leaky integrate-and-fire neuron model with membrane decay controlled by beta.
https://snntorch.readthedocs.io/en/latest/snn.neurons_leaky.html
3. snnTorch documentation: package overview for gradient-based learning with spiking neural networks using PyTorch.
<https://snntorch.readthedocs.io/en/stable/>
4. MNIST handwritten digit database, Yann LeCun, Corinna Cortes, and Christopher J. C. Burges.
<https://yann.lecun.org/exdb/mnist/>
5. UCI Machine Learning Repository: MNIST Database of Handwritten Digits.
<https://archive.ics.uci.edu/dataset/683/mnist+database+of+handwritten+digits>
6. Vite documentation: static asset handling and `public/` directory behavior.
<https://vite.dev/guide/assets>
7. Vite documentation: static deployment and GitHub Pages deployment guidance.
<https://vite.dev/guide/static-deploy>
8. Svelte documentation and project site.
<https://svelte.dev/>
9. D3 documentation: line generator for SVG path data.
<https://d3js.org/d3-shape/line>
10. GitHub Pages documentation: configuring a publishing source for a GitHub Pages site.
<https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-a-github-pages-project>
11. CTAN: ctex package for Chinese typesetting in LaTeX.
<https://ctan.org/pkg/ctex>